NFL Wordle Website — Project Conversation Transcript

Project: NFL Wordle
Framework: Python and Drafter
Course context: CISC 106
Conversation period: November 7–12, 2026

November 7, 2026 — Starting the NFL Wordle Project

1. Choosing the project

User:
I need to create a website based on anything. I chose to make an NFL Wordle game.

Discussion and solution:
We discussed creating a game in which the website selects a mystery NFL player. The user searches for and guesses players, and the game compares information about each guess against the mystery player.

The planned player information included:

- Player name and ID
- Current and previous teams
- Position
- Jersey numbers
- College
- Draft year
- Draft round
- Draft pick number
- Experience
- Statistics

2. Installing NFL data libraries

User:
Can I use `nflreadpy`, `pyarrow`, and `pandas`?

Problem:
The Drafter environment runs in a restricted Python environment, and the user encountered errors such as:

```
ModuleNotFoundError: No module named 'nflreadpy'
```

Similar problems occurred with `pandas` and `pyarrow`.

Solution discussed:
Separate the data-download process from the actual website.

1. Use a regular local Python script to download NFL data using `nflreadpy` and `pandas`.
2. Export the data to a CSV file.
3. Load the CSV in the Drafter website using Python's built-in `csv` module.
4. Convert the CSV rows into the project's dataclasses.

This avoids requiring the Drafter environment to install the NFL-specific libraries.

3. Initial code structure

The project began using dataclasses such as:

- `State`
- `Teamlist`
- `Player_numbers`
- `Stats`
- `Draft`
- `Player`

The website also began with several game routes.

Errors discussed:

- Syntax errors caused by malformed quotation marks.
- Incorrect State initialization, such as `State("")`.
- Missing routes when navigating to a page.

Solution:
Use the correct number and types of fields when initializing `State`, and make sure every route referenced by a button exists.

4. Missing route error

User's issue:
The website reported that a page for `/gamemode1` was missing.

Solution:
Create the route functions before using their paths in buttons. The game was organized around separate game-mode routes, with navigation back to the home page.

November 8–9, 2026 — Building the Game

5. Replacing player text entry with a dropdown

User:
Can you replace the text box with a dropdown list?

Discussion:
A normal dropdown can show a list of players, but a large NFL player list would be difficult to navigate.

User's follow-up:
Is there one where you can type something in and it would filter the list of players down to a few?

Solution discussed:
Create a search function that:

1. Receives the player's search text.
2. Splits the search into name parts.
3. Searches the available player list.
4. Returns matching players.
5. Displays the results in a `SelectBox`.

The search was designed to support:

- First-name searches.
- First and last name searches.
- Partial name matching.
- A limited number of results to keep the dropdown manageable.

6. Comparing player information

User:
Can you tell the user if the player's information is higher or lower than the answer player's information?

Solution:
Create comparison functions for numerical player information.

The intended feedback:

- Correct value: green.
- Guess is lower than the answer: upward arrow.
- Guess is higher than the answer: downward arrow.
- Missing information: N/A.

For example, if the mystery player's draft pick is 20 and the guess player's draft pick is 50, the game indicates that the guess is too high.

7. Organizing the result boxes

User:
Can you make all the boxes in a straight line?

Problem:
A Drafter error occurred:

```
Row() argument after * must be an iterable, not Div
```

Solution discussed:
The layout needed to receive individual components in the correct format. In the later version, the result boxes were placed inside `Div(*boxes)` instead of using the problematic `Row(*boxes)` construction.

November 9–11, 2026 — Improving the Game and Meeting Requirements

8. Reviewing the project requirements

The assignment required:

- At least 7 routes.
- At least 5 Pages, or 10 Fragments, or a combination.

- A State dataclass with at least 4 fields.
- A list or dictionary of class instances stored in State.
- At least 4 State fields meaningfully modified in a route.
- At least 3 input fields.
- At least 3 meaningful conditional statements.
- At least one loop over a list or dictionary of class instances.
- Legitimate website functionality.
- No mutable global variables.
- Use of Drafter.

Discussion:
We reviewed the existing project and identified which requirements were already met and which needed additional implementation.

9. Improving the game difficulty

User:
Is there a way to make it harder?

Discussion:
We considered changing the game rules when Hard Mode is selected.

User's requested changes:

- Reduce the number of guesses.
- Only use a few comparison categories.
- Use draft year, draft number, and position in Hard Mode.

Solution:
Use a `hard_mode` Boolean in the State dataclass.

The intended rules became:

Setting	Normal Mode	Hard Mode
Maximum guesses	6	4
Comparison categories	7	3
Categories	Position, jersey number, draft year, team, experience, college, name	Draft year, draft number, position

The number of guesses is determined by a helper function:

```
None
def max_guesses_for(state: State) -> int:
    if state.hard_mode:
        return 4
    else:
        return 6
```

10. Updating the routes

User:
First do the routes.

Solution:
The game was organized so that Normal Mode and Hard Mode each have their own route.

- Normal Mode sets `hard_mode` to `False`.
- Hard Mode sets `hard_mode` to `True`.
- Both routes start a new game.
- The game logic checks the difficulty setting when comparing guesses.

11. Organizing the guess history

The project used:

```
None
guesses: list[Guess]
```

Each `Guess` object stores:

- The player's name.
- A list of comparison results.

This allows the website to keep the results of multiple guesses and display them in the game history.

12. Avoiding statistics

User:
No statistics.

Decision:
The game would not use a statistics feature. The comparison system would focus on player information rather than a separate statistical system.

November 11, 2026 — Loading NFL Players from CSV

13. Using a CSV file

Problem:
The NFL data libraries were not available in the Drafter environment.

Solution:
Use an already-downloaded CSV file instead.

The project moved toward loading:

`2026 active player(NFL).csv`

The CSV would provide the information needed to create Player objects.

14. Loading the CSV

The code used Python's built-in CSV functionality:

```
None
import csv

def load_players():
    loaded_players = []

    with open("players.csv", newline="") as file:
        reader = csv.DictReader(file)

        for row in reader:
            # Create a Player object
            pass

    return loaded_players
```

The actual implementation was later expanded to create the nested dataclasses.

15. Fixing the `__file__` error

Error:

```
NameError: name '__file__' is not defined
```

Cause:
The environment did not provide `__file__` in the way the code expected.

Solution discussed:
Avoid relying on `__file__` to locate the CSV. Use the correct working directory and open the CSV by its filename.

16. Fixing the Drafter `open()` problem

Error:

```
ValueError: Cannot determine user directory because the path to the main user file is not specified.
```

Cause:
Drafter's `open()` behavior was interfering with ordinary Python file access.

Solution:
Import `builtins` and use the built-in file-opening function explicitly:

```
None
import builtins

with builtins.open(
    "2026 active player(NFL) - Sheet1.csv",
    newline="",
    encoding="utf-8-sig"
) as file:
    reader = csv.DictReader(file)
```

The user was also advised to save the Python file and run the saved script using Thonny's normal Run command rather than running editor content with `%Run -c $EDITOR_CONTENT`.

17. Handling optional numerical values

Some NFL players may not have draft information or other numerical values.

Solution:
Create an `optional_int()` helper that converts valid values to integers and returns `None` for missing or invalid values.

```
None
def optional_int(value: str) -> int | None:
    value = (value or "").strip()

    if value.lower() in ("", "nan", "none", "null"):
        return None

    try:
        return int(float(value))
    except ValueError:
        return None
```

This prevents missing CSV values from crashing the application.

November 12, 2026 — Search Filters, Layout, and Final Fixes

18. Adding a position filter

The player search was expanded to include a position dropdown.

The search function now accepts:

- The list of players.
- The user's search text.
- The selected position.

It filters players by position and name, returning up to 10 results.

Example:

None

```
def search_players(
    players: list[Player],
    search: str,
    position_filter: str
) -> list[str]:
    results = []
    search_parts = search.lower().strip().split()

    for player in players:
        name_parts = player.name.lower().split()

        if not name_parts:
            continue

        if (
            position_filter != "All Positions"
            and player.position.upper() != position_filter.upper()
        ):
            continue

        if not search_parts:
            match = True
        elif len(search_parts) == 1:
            match = name_parts[0].startswith(search_parts[0])
        else:
            match = (
                name_parts[0].startswith(search_parts[0])
                and name_parts[-1].startswith(search_parts[-1])
            )

        if match:
            results.append(
                player.name + " (" + player.position + ")"
            )

        if len(results) == 10:
```

```
            break

    return results
```

19. Fixing duplicate component names

Error:

```
payload.verification_failed: Payload verification failed for the page 'normal_mode':

The page returned from normal_mode has multiple components with the same name: 'player_search' is used by: TextBox, TextBox
```

Cause:
The page was creating two TextBoxes with the same name.

One was created in the newer search-control section, and another was still being added by the old conditional code.

Solution:
Only create one `TextBox("player_search", "")`.

The search controls should all be placed inside one conditional block:

```
None
if not state.game_over:
    content.extend([
        TextBox("player_search", ""),
        SelectBox("position_filter", [...]),
        Button("Search", "/search")
    ])
```

Remove the old duplicate TextBox.

20. Improving position comparisons

The position comparison system was designed to compare broad position groups.

Offense: QB, RB, FB, WR, TE, and offensive line positions.

Defense: DL, DE, DT, NT, EDGE, LB, CB, DB, and safety positions.

Special teams: K, P, and LS.

The comparison logic distinguishes:

- Same group: green.
- Offense versus defense: yellow.
- Special teams versus another group: gray.
- Unknown positions: gray.

This prevents missing or unknown position information from incorrectly appearing as a correct match.

21. Improving the result boxes

The result boxes support:

- Green for correct.
- Yellow for a related category.
- Gray for unavailable or unrelated information.
- Red for incorrect exact matches.

The display function uses:

```
None
return Div(*boxes)
```

This avoids the previous `Row(*boxes)` error.

22. Centering the home page

User:
I want the text to be centered.

Solution:
Apply text alignment to the parent `Div` instead of trying to center each button individually.

Example:

```
None
def index(state: State) -> Page:
    return Page(state, [
        Header("NFL Wordle", style_text_align="center"),
        Div(
            "Guess the mystery NFL player!",
            Button("Normal Mode", "/normal_mode"),
            Button("Hard Mode", "/hard_mode_route"),
            Button("Instructions", "/instructions"),
            style_text_align="center"
        )
    ])
```

This centers the content within the parent container.

23. Final route and game behavior

The latest version included these routes:

1. `index`
2. `gamemode1`
3. `search`
4. `guess`
5. `new_game_route`
6. `normal_mode`
7. `hard_mode_route`
8. `instructions`

The game supports:

- Home page navigation.
- Normal Mode.
- Hard Mode.
- Player search.
- Position filtering.
- Player selection.
- Guess comparison.
- Guess history.
- Win and loss conditions.
- Starting a new game.

- Instructions.

24. Latest State and dataclass structure

The latest version used:

```
None
@dataclass
class Guess:
    player_name: str
    results: list[str]


@dataclass
class State:
    gamemode: int
    answer_id: str
    guesses: list[Guess]
    game_over: bool
    message: str
    hard_mode: bool
    players: list["Player"]
    developer_mode: bool
```

Additional dataclasses:

```
None
@dataclass
class Teamlist:
    current_team: str
    past_team: list[str]


@dataclass
class Player_numbers:
    current_number: int | None
    past_number: list[int]
```

```
@dataclass
class Draft:
    year: int
    draft_round: int | None
    pick_number: int | None
    is_drafted: bool
    team: str


@dataclass
class Player:
    player_id: str
    name: str
    teams: Teamlist
    position: str
    jersey_numbers: Player_numbers
    college: str
    draft: Draft | None
    experience: int | None
```

25. Final known implementation details

- The CSV loader uses `builtins.open()`.
- The search system supports name matching and position filtering.
- Search results are limited to 10 players.
- Normal Mode allows 6 guesses.
- Hard Mode allows 4 guesses.
- Hard Mode uses draft year, draft number, and position.
- The game stores guesses as `Guess` objects.
- The game uses `Player` objects loaded from the CSV.
- The developer answer display is controlled by `developer_mode`.
- `Teamlist.past_team` and `Player_numbers.past_number` are currently empty lists; historical team and jersey-number data are not fully implemented.

Important final setup note:
The latest code still needs to be tested against the actual CSV headers and the installed Drafter

version. The CSV filename and column names must match the loader exactly. The developer mode should be set to `False` before submitting the project.

End of reconstructed transcript

Project status at the last discussion: The NFL Wordle game had its main routes, game modes, CSV loading approach, search/filtering system, and guess comparison logic assembled. The remaining work was to test the complete application, verify the CSV data, and make any final adjustments required by the assignment.

Full Conversation link:https://chatgpt.com/share/6af625b9-67a8-83e9-b80c-6c6a59f73d88